

Architectural Blueprint and Feasibility Analysis: Integrating Meilisearch within the Olam Agri Enterprise Ecosystem

Strategic Context and Enterprise Alignment

The global agricultural supply chain represents one of the most complex, fragmented, and data-intensive operational environments in the modern enterprise landscape. Organizations operating at this scale require instantaneous visibility into highly distributed datasets that track commodities from rural origins to global destinations. Olam Agri, a leading agri-business operating across more than 30 countries, manages the sourcing, processing, and distribution of essential food and agriculture products, including grains, oilseeds, rice, cotton, and rubber.¹ To manage this immense logistical and operational burden, the organization relies on a sophisticated digital infrastructure anchored by SAP S/4HANA Cloud via the RISE with SAP initiative, hosted primarily on Amazon Web Services (AWS).² This core Enterprise Resource Planning (ERP) system acts as the immutable system of record for the organization's financial, logistical, and operational data.

However, the modern enterprise ecosystem extends far beyond the core ERP. Olam Agri operates specialized, user-facing digital platforms designed to interact directly with external stakeholders. Notable among these is Olam Direct, an industry-leading mobile application that empowers over 70,000 smallholder farmers across 12 countries to bypass intermediaries, access real-time market prices, secure financing, and directly transact with the supply chain.⁴ Additionally, the organization operates AtSource, a revolutionary sustainability insights platform that tracks over 350 environmental and social metrics—such as carbon footprints and water usage—providing corporate customers with unprecedented transparency into the origins of their agricultural products.⁵

The technological stewardship of this diverse ecosystem is driven by Mindsprint, Olam's dedicated technology spin-off, which leverages SAP Business Technology Platform (BTP), artificial intelligence (AI), and advanced middleware to modernize enterprise operations and engineer purpose-built digital solutions.⁷ Within this highly integrated, high-volume environment, the requirement for instantaneous, typo-tolerant, and highly relevant search capabilities across user-facing applications is paramount. Business users analyzing sustainability dashboards and rural farmers querying crop prices require search responses measured in milliseconds, even under constrained network conditions.

Traditional backend search engines and relational databases, while exceptionally powerful for deep analytics and transactional integrity, frequently introduce latency and operational complexity that degrade the end-user experience in front-facing applications.¹¹ Meilisearch, an

open-source, Rust-based search engine, has emerged as a highly specialized solution for instant-search experiences, engineered to consistently deliver search responses in under 50 milliseconds.¹² Despite its performance advantages, introducing a new datastore into an SAP-centric, enterprise-grade architecture raises critical questions regarding systemic performance degradation, resource monopolization, integration patterns, hosting topologies, and architectural blockers. This report provides an exhaustive technical analysis of deploying Meiliseach within the Olam Agri technology stack, evaluating its systemic impact, defining strict integration pathways with SAP S/4HANA, and outlining the hardware and software prerequisites for a resilient, production-grade deployment.

Systemic Performance Dynamics and Workload Impact

A primary architectural concern when introducing a high-performance search engine into an existing technological ecosystem is the potential for performance degradation on the host infrastructure and the collateral impact on co-located applications. The specific query—whether "performance will be down" due to the implementation of Meiliseach—requires a granular examination of the engine's underlying storage mechanics, memory management profile, and concurrency models. The definitive answer is that Meiliseach can and will degrade the performance of other applications if it is improperly co-located on shared computing resources without strict isolation protocols.¹³

Memory-Mapped Storage and the LMDB Engine

Meiliseach is fundamentally built upon the Lightning Memory-Mapped Database (LMDB), a highly optimized, transactional key-value store renowned for its read performance.¹³ Unlike Java Virtual Machine (JVM)-based search engines such as Elasticsearch, which rely heavily on allocated heap memory and continuous garbage collection cycles, LMDB operates via memory-mapped files (mmap).¹⁵ This architectural decision delegates the caching of database pages entirely to the operating system's virtual memory manager.¹⁷

The operational performance of Meiliseach is intrinsically linked to the ratio of available physical Random Access Memory (RAM) to the total database size on disk.¹⁸ Because Meiliseach utilizes a memory map, the system achieves maximum performance when the OS can fit the entire active dataset into RAM, effectively eliminating disk I/O during search queries.¹⁴ If a dataset is vast but RAM is constrained, the operating system is forced into a state of thrashing—frequently paging data in and out of the slower disk storage—which results in severe latency spikes and degraded search speeds.¹⁹

However, empirical evidence indicates that a strict 1:1 RAM-to-disk ratio is rarely necessary in production. A ratio of approximately $\frac{1}{3}$, or even $\frac{1}{10}$, does not materially impact performance for many standard workloads, provided that the underlying storage utilizes

high-speed NVMe Solid State Drives (SSDs).¹⁶ Because Meilisearch delegates memory management to the OS kernel, system monitoring tools such as `htop` or `top` will frequently display near-maximum RAM utilization, often highlighted by a large caching footprint.¹⁷ This observation frequently alarms system administrators who misinterpret the metric as a memory leak; in reality, it reflects the OS aggressively utilizing all available free memory to populate the page cache and accelerate read operations.¹⁷ The OS will automatically evict these cached pages if other critical applications require physical memory, thereby minimizing the risk of out-of-memory (OOM) failures strictly from read operations.¹⁶

CPU Utilization, Multi-Threading, and Indexing Overhead

The primary vector for performance degradation stems from the asymmetry between how Meilisearch handles read (search) operations versus write (indexing) operations. Search queries operate strictly on a single thread.¹⁴ This deliberate design choice prevents individual, highly complex queries from monopolizing the CPU and starving concurrent requests, ensuring a predictable, low-variance latency profile.¹⁴

Conversely, indexing is a highly intensive, multi-threaded, and memory-consuming operation.²⁰ When new documents are ingested from an upstream system like SAP S/4HANA, the Meilisearch indexer must compute document changes, extract filterable facets, tokenize text, and build complex word-proximity matrices for phrase and attribute ranking.²¹ To accomplish this efficiently, Meilisearch utilizes the Rust `sysinfo` library to automatically detect the host machine's resources, subsequently adapting its behavior to consume a maximum of two-thirds (66%) of the available RAM and up to half of the available CPU cores.²⁰

For example, on a 12-core server, the background indexer will aggressively utilize six cores at near 100% capacity to process B-tree updates in parallel.²⁰ While this automated resource capping ensures that Meilisearch remains responsive to incoming search queries during massive data updates, it introduces significant CPU spikes and immense disk I/O pressure to the host machine.¹⁷ If Meilisearch is deployed on a shared virtual machine or physical server hosting other critical applications—such as a localized SAP gateway, an API orchestration layer, or a database—the intensive indexing process will inevitably monopolize CPU cycles, thrash the OS page cache, and cause severe performance degradation for those co-located services.¹³

To mitigate this systemic risk, enterprise architects must enforce strict resource isolation. Meilisearch should be deployed on dedicated compute instances.¹³ If co-location is absolutely unavoidable, system administrators must utilize the `--max-indexing-memory` and `--max-indexing-threads` configuration flags to enforce hard limits on the engine's resource consumption, purposefully trading indexing speed for host stability.²⁰

Comparative Performance Analysis: The Search Landscape

To properly contextualize Meilisearch's operational profile and justify its deployment over

incumbent solutions within the Olam Agri architecture, it must be compared against established enterprise alternatives, primarily Elasticsearch and Algolia.¹² The search landscape is defined by a spectrum of trade-offs between managed convenience, operational control, and architectural complexity.¹²

Architectural Vector	Meilisearch	Elasticsearch	Algolia
Core Engine Technology	Rust-based, LMDB. ¹⁴	Java-based, Lucene JVM. ¹⁵	Proprietary C++ distributed engine. ¹⁵
Hosting Model	Open-source (CE) / Source-available (EE). ¹⁵	Elastic License / Open-source forks. ¹⁵	Fully managed SaaS (Closed source). ¹²
Latency Profile	Consistently <50ms. ¹²	Highly variable based on JVM tuning. ¹²	Sub-100ms globally via Distributed Search Network. ¹²
Scaling Architecture	Single-node optimized; EE requires explicit sharding. ²⁵	Distributed, multi-node clustered topology by default. ¹⁵	Globally distributed SaaS network. ¹²
Primary Use Case	Instant, typo-tolerant end-user frontend search. ¹¹	Deep analytics, log processing, complex aggregations. ¹¹	High-budget e-commerce, zero-overhead enterprise search. ¹²
Operational Overhead	Minimal; single binary deployment. ²⁹	High; requires dedicated JVM and shard management. ¹¹	Zero; fully managed infrastructure. ¹²
Indexing Velocity	Exceptionally fast (up to 7x faster than Lucene on moderate datasets). ²³	Moderate, but scales infinitely horizontally. ¹⁵	High-speed real-time indexing pipeline. ²³

For an organization like Olam Agri, utilizing Elasticsearch to power a mobile application like

Olam Direct would introduce substantial, often unnecessary operational overhead.¹¹ Elasticsearch is an analytics powerhouse, but optimizing its Java heap allocations, managing shard rebalancing, and mastering its complex Query DSL merely to provide a fast, typo-tolerant search bar is an architectural misallocation of resources.¹¹ Conversely, Algolia provides zero operational overhead and excellent performance but introduces strict vendor lock-in and high volumetric costs that scale prohibitively with the massive datasets typical of global agricultural supply chains.¹²

Meilisearch strikes a deliberate architectural balance.¹² It offers the developer-friendly API and sub-50ms latency profile of Algolia, combined with the self-hosting flexibility and open-source economics of Elasticsearch, provided that the dataset fits within the architectural constraints of a single-node memory map or a managed sharded cluster.¹²

Hardware and System Requirements

Deploying Meilisearch within a production environment demands strict adherence to hardware specifications uniquely tailored to the LMDB storage engine. Unlike traditional relational databases that prioritize compute power for query planning, Meilisearch's hardware dependencies are heavily skewed toward storage velocity and memory capacity.¹⁴

Storage, Memory, and Compute Specifications

The hardware requirements for self-hosting Meilisearch scale dynamically based on the volume of the dataset, the number of configured searchable attributes, and the concurrent query volume.¹⁸

1. **Storage Media (The Critical Bottleneck):** Because Meilisearch relies entirely on memory mapping, the latency of disk read operations directly dictates search speed whenever data pages are evicted from RAM.¹⁶ Deploying Meilisearch on Hard Disk Drives (HDDs), Network File Systems (NFS), or high-latency network-attached storage is entirely unacceptable and will result in catastrophic performance degradation.¹⁶ Production deployments must utilize high-performance NVMe Solid State Drives (SSDs) or enterprise-grade block storage (e.g., AWS EBS io2 Block Express) to ensure microsecond disk lookup times.¹⁴
2. **Storage Volume Sizing:** The allocated disk space must be vastly larger than the raw JSON payload ingested from the upstream system. As a heuristic, the storage volume should be at least ten times the size of the raw dataset.¹⁹ This expansion occurs because Meilisearch generates highly complex inverted indexes, word proximity matrices, and facet dictionaries to guarantee rapid retrieval.¹⁹ Furthermore, LMDB requires significant free space to manage data fragmentation, as deletions do not immediately return physical space to the OS.¹⁶
3. **Memory (RAM):** The total physical RAM must be sufficient to hold the most frequently accessed data structures. While the engine will not crash if the database size exceeds

physical RAM, search latency will degrade proportionally to the frequency of disk page faults.¹⁶ A general guideline dictates that the machine should have enough RAM to hold the full dataset in memory during intensive indexing operations to prevent out-of-memory kernel panics.²⁰

4. **Processor (CPU):** Processor selection must balance two opposing operational requirements. Because search queries operate sequentially on a single thread to minimize latency variance, processors boasting exceptionally high single-core clock speeds (e.g., Intel Xeon or AWS Graviton processors optimized for compute-intensive workloads) yield the best query performance.¹⁴ Conversely, the background indexer scales linearly with core count.¹⁴ A balanced multi-core CPU allows the engine to dedicate isolated logical cores to the highly intensive indexing B-tree updates while reserving high-frequency cores to serve concurrent user queries without interruption.¹⁹

Dataset Size (Raw JSON)	Estimated DB Size on Disk	Minimum Recommended RAM	Storage Requirement	Compute Requirement
< 1 GB	~10 - 15 GB	8 GB	50 GB NVMe SSD	4 vCPU (High Clock Speed)
1 GB - 10 GB	~100 - 150 GB	32 GB - 64 GB	500 GB NVMe SSD	8 vCPU
10 GB - 50 GB	~1 TB - 1.5 TB	128 GB - 256 GB	2 TB NVMe SSD	16+ vCPU
> 50 GB	> 1.5 TB	512 GB+	5 TB+ NVMe SSD	32+ vCPU (or Sharded Cluster)

(Note: Database expansion ratios vary significantly based on the number of searchableAttributes and filterableAttributes configured in the index settings.¹⁸)

Self-Hosting Options and Deployment Topologies

Meilisearch offers a highly versatile deployment model, distributed as a single compiled Rust

binary with no external dependencies (unlike Elasticsearch, which requires a JVM).²⁹ This architecture supports multiple self-hosting topologies ranging from bare-metal installations to sophisticated Kubernetes orchestrations.

Bare Metal and Virtual Machine Deployments

For maximum raw performance, executing the standalone Meilisearch binary directly on a Linux distribution (such as Debian 12) removes all hypervisor and containerization abstraction overhead.³² This is often the preferred method for highly latency-sensitive deployments.

In this configuration, system administrators must implement fundamental security isolation protocols. Running the Meilisearch process as the root user exposes the host to unnecessary security vulnerabilities.³² A dedicated system user with restricted shell access (`useradd -d /var/lib/meilisearch -s /bin/false -m -r meilisearch`) must be created to own the binary and the localized data directories (`/var/lib/meilisearch/data`).³²

To ensure high availability and stability, the binary should be managed as a background service via `systemd` in Linux.³² This guarantees that the process launches automatically upon OS boot and restarts autonomously upon failure (`Restart=on-failure`), significantly reducing downtime.³² Furthermore, because Meilisearch communicates exclusively via a REST API, processing external requests directly over HTTP represents a major security flaw.³² A reverse proxy, such as Nginx, must be configured in front of the Meilisearch instance to handle TLS termination using Let's Encrypt / Certbot, ensuring end-to-end encryption of all search queries and data ingestion payloads.²⁴

Docker and Containerization

Containerized deployment provides unparalleled environmental consistency, ensuring the application behaves identically across development, staging, and production environments.³³ Meilisearch publishes official, version-tagged Docker images to Docker Hub (`getmeili/meilisearch:latest`).³³

When launching the engine via Docker (`docker run`), it is absolutely critical to mount persistent host volumes (`-v $(pwd)/meili_data:/meili_data`) to the container.³³ Failure to bind a persistent volume will result in the catastrophic loss of all indexed data the moment the container is terminated or updated.³³ Within a Docker environment, configuring the instance is typically handled by injecting environment variables at runtime.³³

Cloud Infrastructure: AWS and Nutanix

Given Olam Agri's reliance on Amazon Web Services (AWS) and its extensive utilization of Nutanix hyperconverged infrastructure (HCI) for virtual desktop infrastructure (VDI) and file storage, both platforms present viable hosting targets.³

AWS Deployment: Meilisearch offers pre-configured Machine Images via the AWS Marketplace, allowing for rapid deployment of compute instances optimized specifically for the search engine's workload.³⁶ When deploying on AWS, selecting EC2 instances from the memory-optimized or storage-optimized families (e.g., R6id or I4i instances), paired with io2 Block Express EBS volumes for guaranteed IOPS, is highly recommended to satisfy the rigorous disk latency requirements of the LMDB engine.¹⁶

Nutanix Deployment: Olam Agri has successfully deployed Nutanix Cloud Infrastructure, including the Acropolis Hypervisor (AHV) and Nutanix Unified Storage, to reduce infrastructure costs and support highly scalable workloads.³⁵ Deploying Meilisearch atop Nutanix AHV is highly feasible.³⁹ The software-defined, scale-out storage solution of Nutanix provides the necessary high-throughput, low-latency block storage required by Meilisearch's memory map without forcing architectural compromises.³⁵ Furthermore, Nutanix allows for seamless vertical scaling of VMs, enabling IT administrators to dynamically allocate additional RAM to the Meilisearch instance as the database size inevitably expands.³⁵

Configuration Methodologies

Regardless of the hosting topology, Meilisearch prioritizes configuration via three hierarchical methods, allowing for flexible infrastructure-as-code deployments.³⁴

1. **Command-Line Options:** Flags passed directly to the binary during execution (e.g., `./meilisearch --db-path ./meilifiles --http-addr 'localhost:7700'`). These take the absolute highest precedence.³⁴
2. **Environment Variables:** System variables prefixed with `MEILI_` (e.g., `MEILI_MASTER_KEY`). This is the preferred method for Docker and Kubernetes deployments.³³
3. **Configuration File:** A structured `.toml` file (`config.toml`) that can be easily version-controlled in Git repositories, defining comprehensive instance behaviors.³⁴

Community Edition vs. Enterprise Edition

A critical decision point for Olam Agri involves selecting the appropriate licensing and feature tier. The Meilisearch Community Edition (CE) is fully open-source under the MIT license and contains all core search algorithms, including full-text capabilities, hybrid semantic search, and AI-powered vector capabilities.¹⁵ It is perfectly suited for independent hosting of moderate-to-large datasets.

However, the Meilisearch Enterprise Edition (EE), governed by the Business Source License (BUSL), unlocks advanced clustering capabilities required for mission-critical, massive-scale deployments.⁴⁰ The singular, defining feature exclusive to the Enterprise Edition is **sharding**.⁴⁰ Sharding allows massive indexes—which would otherwise exceed the storage and memory capacity of a single server—to be horizontally partitioned across multiple interconnected

nodes.²⁵

Furthermore, the Enterprise Edition enables data replication, duplicating these shards across instances to guarantee high availability and failover continuity if a primary node terminates.²⁵ For a global supply chain organization like Olam Agri, if a centralized product catalog, logistics database, or multi-tenant analytics index exceeds the terabyte threshold, the Enterprise Edition transitions from a luxury to an architectural necessity to maintain the sub-50ms latency service level agreement (SLA).²⁴ Self-hosting the EE in a production environment requires a direct commercial license agreement with Meilisearch.⁴²

Architectural Integration with the Olam Agri Ecosystem

Integrating a high-speed search index into an enterprise architecture dominated by SAP S/4HANA requires robust middleware pipelines, precise API orchestration, and resilient data synchronization strategies. Olam Agri's operations are deeply embedded in SAP S/4HANA Cloud via the RISE with SAP program, orchestrated by Mindsprint.² Within this framework, SAP S/4HANA acts as the immutable system of record, while Meilisearch functions as a highly denormalized, read-optimized "sidecar" projection of that data, designed exclusively to serve frontend applications.

Backend Data Extraction from SAP S/4HANA

Synchronizing data from the highly normalized, deeply relational tables of the SAP HANA in-memory database into the flattened, document-oriented JSON structure required by Meilisearch is the primary integration challenge.

Historically, SAP data extraction relied on legacy ABAP extractors and complex Remote Function Calls (RFCs).⁴³ However, in modern SAP S/4HANA Cloud environments, the Operational Data Provisioning (ODP) framework is the standard mechanism for data extraction and replication.⁴³ The ODP framework allows enterprise developers to expose SAP Core Data Services (CDS) views—which act as a virtual data model atop the transactional tables—as standard, RESTful OData APIs.⁴³

To extract data representing agricultural commodities, logistics tracking updates, or smallholder farmer profiles, Olam Agri's engineering teams must navigate to the SAP Reference IMG (transaction SPRO or /n/IWFND/MAINT_SERVICE) to activate the OData channel for the specific CDS views.⁴⁵ This creates an inbound HTTP endpoint that external systems can query to retrieve SAP data.⁴⁷

A critical operational consideration during this extraction process is volumetric constraints. SAP OData APIs will typically time out, consume excessive application server memory, or fail outright

if queried for millions of records in a single, synchronous HTTP request.⁴⁸ Consequently, the integration layer must implement strict **client-side paging**.⁴⁸ By utilizing OData query parameters such as \$skip and \$top, the middleware can request records in manageable chunks (e.g., 10,000 records at a time).⁴⁸ Furthermore, executing full reloads of the entire SAP database on a nightly basis is highly inefficient. The ODP framework supports delta queues, enabling the middleware to request only the specific records that have been created, updated, or deleted since the last synchronization event, vastly reducing network payload sizes and processing time.⁴³

Middleware and Event-Driven Orchestration via SAP BTP

To manage this complex extraction, transformation, and loading (ETL) process, Olam Agri utilizes the SAP Business Technology Platform (BTP) and the SAP Integration Suite as its central nervous system.⁸ SAP Integration Suite provides an enterprise-grade Integration Platform as a Service (iPaaS) capable of seamlessly orchestrating the data flow between SAP S/4HANA and external REST APIs like Meilisearch.⁵⁰

Using SAP Cloud Integration (a core component of the Suite), developers can design graphical integration flows (iFlows).⁵⁰ These iFlows can be scheduled to trigger periodically or listen for business events. The iFlow executes the paginated calls to the SAP OData APIs, maps the complex XML or nested JSON payloads into the flat JSON array structures required by Meilisearch, and executes asynchronous HTTP POST requests to the Meilisearch /documents endpoint.⁵⁰

Alternatively, custom microservices can be developed and deployed directly on SAP BTP using the SAP Cloud SDK for Java or Python.⁵³ The SDKs abstract the complexities of OData communication, authentication, and destination management.⁵³ For example, the Java SDK provides a fluent, type-safe API to consume SAP services:

Java

```
final List<Product> products = new DefaultProductMasterService()
    .getAllProduct()
    .select(Product.PRODUCT, Product.CREATION_DATE)
    .filter(Product.PRODUCT_TYPE.eq("1"))
    .top(10)
    .execute();
```

Once the data is retrieved from SAP using this methodology, the microservice utilizes the official Meilisearch Java or Python SDKs to ingest the batch into the search index.⁵⁵ For organizations seeking low-code alternatives, third-party connectors—such as CData’s JDBC/ODBC drivers for Elasticsearch/REST or MuleSoft’s SAP S/4HANA connector—can also facilitate this pipeline by bridging the SAP application layer directly to the search engine’s API.⁵⁷

Ingestion Optimization Tactics

Regardless of the middleware utilized, the methodology of data ingestion dramatically impacts Meilisearch's stability. Meilisearch performs optimally when documents are sent in large, compressed batches rather than as thousands of individual HTTP requests.⁵² Transmitting 100,000 documents in two batches of 50,000 is exponentially faster for the LMDB writer thread than sending four batches of 25,000.⁵² All HTTP payloads transmitted from SAP BTP to Meilisearch must utilize gzip, deflate, or br (brotli) compression to minimize network bandwidth and transfer latency.³¹

Crucially, Meilisearch's indexing speed is directly proportional to the number of configured searchableAttributes and filterableAttributes.²¹ Unlike Elasticsearch, where "index everything" is a common (if deeply flawed) practice, Meilisearch requires extreme precision. Developers must strictly curate the searchable attributes list to include only the fields that end-users will explicitly query (e.g., crop type, location, ID).³¹ Excluding massive text blobs from this list prevents the engine from generating unnecessary word-proximity matrices, thereby saving massive amounts of RAM and vastly accelerating the background indexer.²¹

Frontend Integration: Empowering Olam Direct and AtSource

The ultimate objective of deploying a high-performance search engine is to empower frontend applications. Olam Agri operates several highly specialized user interfaces that stand to benefit immensely from Meilisearch's capabilities.

Olam Direct: This platform acts as a critical lifeline, connecting over 70,000 smallholder farmers directly to the supply chain via mobile applications.⁴ Farmers utilize the app to check real-time commodity prices, negotiate direct sales to Olam, and request logistics support.⁴ Mobile applications in this domain are typically built using cross-platform frameworks such as Flutter (Dart) or React Native (JavaScript) to maximize reach across disparate device ecosystems.⁶⁰ Meilisearch provides officially supported SDKs for both Dart and JavaScript.⁵⁵ By integrating the Meilisearch client directly into the mobile application, Olam Direct can offer instantaneous, typo-resistant searches for crop advisory content or pricing tables, drastically improving the user experience for individuals operating in rural environments with intermittent or high-latency network connectivity.⁴

AtSource: Olam’s B2B sustainability platform provides corporate customers with supply chain transparency, tracking over 350 environmental and social metrics.⁵ Business users interacting

with complex web dashboards require the ability to rapidly filter and aggregate data across numerous dimensions (e.g., carbon emissions per ton, water usage per farm, geographic origin).⁵ Meilisearch excels at this specific workload. Through its frontend integrations—specifically the instant-meilisearch client—developers can seamlessly bridge Meilisearch with popular UI libraries like InstantSearch.js in React or Vue applications.⁵⁵ This integration enables powerful faceted navigation, dynamic multi-select filtering, and near-instant dashboard updates as users toggle sustainability parameters, providing a fluid, highly responsive analytical experience.⁵⁵

Multi-Tenancy, Security, and Compliance

Olam Agri handles highly sensitive data across diverse legal entities, regional suppliers, and corporate customers. Ensuring strict data isolation is a critical compliance mandate, particularly for platforms like AtSource where competing corporate entities may be utilizing the same underlying infrastructure to view their respective supply chains.

The standard instinct when utilizing relational databases is to create separate schemas, or in search engines like Elasticsearch, separate indexes for each individual tenant.⁶² In Meilisearch, creating thousands of distinct indexes to achieve multi-tenancy is an architectural anti-pattern.⁶² Because the Meilisearch engine processes indexing tasks strictly sequentially—processing one index after another—maintaining hundreds of separate indexes completely destroys indexation throughput, as the engine cannot parallelize tasks across different indexes.⁶²

The mandated approach for multi-tenancy in Meilisearch is to place all documents belonging to all tenants into a single, unified massive index, appending a distinct `company_id` or `tenant_id` attribute to each document.⁶² Strict access control is then enforced dynamically at the application layer using **Tenant Tokens**.⁶³

A Tenant Token is a small, encrypted JSON Web Token (JWT) that presents cryptographic proof that a user is authorized to access specific subsets of an index.⁶⁵ The workflow operates as follows: when a user logs into the AtSource frontend, the frontend requests an access token from the backend server (e.g., a secure SAP BTP microservice).⁶⁵ The backend server utilizes the Meilisearch Master Key to dynamically generate a short-lived Tenant Token, injecting specific search rules into the payload (e.g., `{"filter": "company_id = 'Nestle'"}`).⁶⁵ The backend returns this token to the frontend client, which then queries the Meilisearch API directly using the token.⁶⁵ Because the search rules are cryptographically signed within the token, the client cannot bypass the filter, ensuring total data isolation without degrading the performance of the underlying LMDB engine.⁶³ This Role-Based Access Control (RBAC) methodology is vital for maintaining enterprise compliance standards, such as SOC 2, which are natively supported by the Meilisearch Enterprise infrastructure.²⁴

Architectural Blockers and Enterprise Limitations

While Meilisearch is exceptionally performant, its architectural design relies on strict, hard-coded limitations to guarantee speed and prevent unbounded memory consumption. Enterprise architects must account for these fundamental blockers during the system design phase, as they represent hard limits that cannot be circumvented through configuration.

Hard Data Structure Limitations

1. **Attribute Caps per Document:** Meilisearch strictly enforces a maximum limit of 65,536 attributes per document.¹³ If the integration middleware attempts to ingest a JSON document containing more than 65,536 fields, the engine will throw an error and reject the payload.¹³ This limit is enforced because overly large internal data structures result in exponentially massive databases on disk, leading to severe performance degradation during search operations.¹³
2. **Primary Key Constraints:** The unique identifier (primary key) of a document is stored directly as an LMDB key.¹³ LMDB limits the size of these keys to exactly 511 bytes.¹³ If SAP S/4HANA generates long, compound primary keys (e.g., concatenating plant IDs, sales organizations, and document numbers), the middleware must hash (e.g., SHA-256) or safely truncate these keys before ingestion to prevent the indexing task from failing.¹³
3. **Query Word Processing Limits:** To prevent combinatorial explosions during prefix matching and typo-tolerance calculations, Meilisearch enforces a hard limit on full-text search terms: it ignores all search terms after the 10th word in a query.¹³ Consequently, Meilisearch is not natively designed for long-form, full-paragraph document matching using traditional keyword methodologies.¹³ (Note: This limitation applies to standard text search; utilizing Meilisearch's newer AI-powered vector search capabilities bypasses this limit by relying on semantic embeddings²⁴).
4. **Attribute Position Limits:** For relevancy scoring, the engine indexes a maximum of 65,535 word positions per attribute.¹³ If an attribute contains a massive block of unstructured text, any words occurring after this specific position limit are silently ignored during tokenization and will not yield search hits.¹³

Write Workload Degradation and File Ballooning

A critical architectural blocker for highly dynamic, transactional environments is how Meilisearch handles document deletions and updates via the LMDB engine. When a document is deleted from a Meilisearch index, LMDB updates its internal bitmaps to mark that physical disk space as "free" internally.¹⁶ However, it *does not* return this reclaimed space to the host operating system.¹⁶

This design choice was implemented to avoid the massive CPU overhead associated with periodic database compaction and garbage collection, allowing for consistently high write speeds.¹⁶ However, the consequence is that the physical file size of the database on disk will

strictly increase over time, ballooning continuously as new data is written.¹⁶ In an enterprise scenario like Olam Agri, where millions of agricultural tracking records are continuously ingested, updated, and deleted daily, this ballooning effect increases pressure on the filesystem cache and the underlying I/O subsystem.⁶⁶ If the write workload is exceptionally high, the constant decoding and encoding of deletion bitmaps within the LMDB B-tree will eventually throttle indexing speed and drive up disk latency.⁶⁶ To mitigate this, systems administrators must implement maintenance windows to periodically export the dataset via snapshots or dumps and perform a clean re-import to physically reclaim disk space.

Synchronous Reindexing Triggers

Furthermore, enterprise teams must implement strict change-management protocols regarding index configuration. Any modification to core index settings—such as adding a new field to the `searchableAttributes` or `filterableAttributes` list—triggers a complete, synchronous re-indexing of all documents within that index.³¹ In a multi-gigabyte dataset, this operation locks the indexer and consumes vast CPU resources for an extended period.⁵² To prevent search latency spikes, schema modifications must be executed strictly during low-traffic maintenance windows.⁵²

For massive datasets utilizing AI-powered vector search (e.g., indexing over 1 million documents with high-dimensional embeddings), the RAM requirement for vector storage can become prohibitive. To counter this, Meilisearch provides a binary quantization feature, which compresses vectors to reduce semantic search precision slightly in exchange for vastly improved indexing speed and reduced storage overhead.⁵² However, this quantization is irreversible; recovering the original vector precision requires re-vectorizing the entire index from scratch.⁵²

Looking toward the future, the Meilisearch 2026 product roadmap aims to address the compute waste associated with maintaining warm indexes that receive infrequent queries.⁴¹ The introduction of "serverless indexes" will allow inactive tenant data to be offloaded to cheap object storage (like AWS S3) and rapidly spun back up upon query arrival, fundamentally altering the unit economics of massive-scale multi-tenant deployments.⁴¹

Strategic Recommendations and Conclusion

Deploying Meilisearch within the Olam Agri enterprise architecture presents a highly compelling opportunity to bridge the fundamental gap between SAP S/4HANA's rigorous transactional integrity and the modern end-user demand for sub-50ms data retrieval in critical applications like Olam Direct and AtSource. While traditional search engines like Elasticsearch offer unmatched analytical depth, their operational complexity and latency variance render them suboptimal for instant-search frontend experiences. Meilisearch provides the necessary developer ergonomics, speed, and typo-tolerance out of the box.

However, a successful integration requires strict adherence to the architectural constraints

inherent in memory-mapped storage systems. To ensure enterprise-grade stability and prevent the search engine from degrading the performance of co-located applications, the following strategic directives must be enforced:

1. **Strict Infrastructure Isolation:** Meilisearch must never be deployed on shared application servers.¹³ It must be provisioned on dedicated, highly optimized infrastructure—such as AWS EC2 memory-optimized instances backed by io2 Block Express volumes, or isolated VMs on Nutanix AHV.³⁶ Sufficient physical RAM must be allocated to hold the active memory map, preventing disk thrashing.¹⁶
2. **Optimized Middleware Pipelining:** Data extraction from SAP S/4HANA must leverage the ODP framework and OData APIs via SAP Integration Suite.⁴⁴ The middleware must implement client-side paging to navigate SAP volumetric limits and employ batched, compressed HTTP payloads to maximize Meilisearch indexing throughput.⁴⁸
3. **Unified Multi-Tenancy Architecture:** To satisfy compliance and data isolation requirements for platforms like AtSource without destroying indexing performance, engineering teams must abandon multi-index architectures. All data must reside in a unified index, secured dynamically via cryptographically signed Tenant Tokens generated by the SAP BTP backend.⁶²
4. **Schema Curation and Change Management:** Developers must ruthlessly curate the searchableAttributes list to minimize RAM consumption and prevent database ballooning.³¹ Configuration changes must be strictly regulated to prevent inadvertent, massive re-indexing events during peak operational hours.⁵²

By meticulously designing the integration pipeline and respecting the hard limits of the LMDB engine, Mindsprint and Olam Agri can successfully synchronize the deep, relational intelligence of SAP S/4HANA with the lightning-fast retrieval capabilities of Meilisearch. This architectural synthesis will fundamentally elevate the digital experience for thousands of global stakeholders, ensuring that critical agricultural data is accessible instantaneously, resiliently, and securely across the global supply chain.

Works cited

1. Building a Brighter Future - Olam Agri, accessed March 30, 2026, <https://www.olamagri.com/content/dam/olam-agri/assets/webp/au/au-pdfs/factsheet/olamagri-corporate-factsheet-2025-english.pdf>
2. Olam Agri Accelerates AI Journey with RISE with SAP on AWS, accessed March 30, 2026, <https://news.sap.com/sea/2024/09/olam-agri-accelerates-ai-journey-with-rise-with-sap-on-aws/>
3. Olam Agri revs up AI push through RISE with SAP on AWS | Frontier Enterprise, accessed March 30, 2026, <https://www.frontier-enterprise.com/olam-agri-revs-up-ai-push-through-rise-with-sap-on-aws/>
4. Olam Direct, accessed March 30, 2026,

<https://www.olamgroup.com/sustainability/innovation-technology/farmgate/olam-direct.html>

5. AtSource with Olam, accessed March 30, 2026, <https://ga-atsource.olamnet.com/>
6. Innovation & Technology - Olam Group, accessed March 30, 2026, <https://www.olamgroup.com/sustainability/innovation-technology.html>
7. Unlocking value, exploring new horizons - Olam Group, accessed March 30, 2026, https://www.olamgroup.com/content/dam/olamgroup/investor-relations/ir-library/annual-reports/annual-reports-pdfs/2024/olam_annual_report_2024_strategic_report.pdf
8. Enterprise application & systems | Mindsprint, accessed March 30, 2026, <https://www.mindsprint.com/services/enterprise-application-and-systems>
9. Mindsprint: Digital transformation & automation solutions, accessed March 30, 2026, <https://www.mindsprint.com/>
10. Mindsprint Taps Grow with SAP to Streamline Global Operations and Deepen Partner Expertise, accessed March 30, 2026, <https://news.sap.com/sea/2025/07/mindsprint-taps-grow-with-sap-to-streamline-global-operations-and-deepen-partner-expertise/>
11. Comparison to alternatives - Meilisearch Documentation, accessed March 30, 2026, <https://www.meilisearch.com/docs/resources/comparisons/alternatives>
12. Algolia vs Elasticsearch: Which Search Fits Your Needs? - Meilisearch, accessed March 30, 2026, <https://www.meilisearch.com/blog/algolia-vs-elasticsearch>
13. Known limitations - Meilisearch Documentation, accessed March 30, 2026, https://www.meilisearch.com/docs/resources/help/known_limitations
14. Recommended system requirements · Issue #374 · meilisearch/meilisearch - GitHub, accessed March 30, 2026, <https://github.com/meilisearch/meilisearch/issues/374>
15. Compare Algolia vs ElasticSearch vs Meilisearch vs Typesense, accessed March 30, 2026, <https://typesense.org/typesense-vs-algolia-vs-elasticsearch-vs-meilisearch>
16. Storage - Meilisearch Documentation, accessed March 30, 2026, <https://www.meilisearch.com/docs/learn/engine/storage>
17. Abnormally high CPU usage compared to RAM · Issue #2581 - GitHub, accessed March 30, 2026, <https://github.com/meilisearch/meilisearch/issues/2581>
18. FAQ - Meilisearch Documentation, accessed March 30, 2026, <https://www.meilisearch.com/docs/resources/help/faq>
19. FAQ - Meilisearch Documentation, accessed March 30, 2026, <https://www.meilisearch.com/docs/learn/resources/faq>
20. Impact of RAM and multi-threading on indexing performance - Meilisearch Documentation, accessed March 30, 2026, https://www.meilisearch.com/docs/resources/self_hosting/performance/ram_multithreading
21. Optimize indexing performance with batch statistics - Meilisearch Documentation, accessed March 30, 2026, https://www.meilisearch.com/docs/capabilities/indexing/tasks_and_batches/optimize_batch_performance

22. CPU Usage Very High · Issue #2027 · meilisearch/meilisearch - GitHub, accessed March 30, 2026, <https://github.com/meilisearch/meilisearch/issues/2027>
23. Navigating Search Solutions: A Comprehensive Comparison Guide to Meilisearch, Algolia, and Elasticsearch - DEV Community, accessed March 30, 2026, <https://dev.to/tioluwani/navigating-search-solutions-a-comprehensive-comparison-guide-to-meilisearch-algolia-and-o0n>
24. Enterprise-grade search solution | Built for growth - Meilisearch, accessed March 30, 2026, <https://www.meilisearch.com/solutions/enterprise>
25. Replication and sharding - Meilisearch Documentation, accessed March 30, 2026, https://www.meilisearch.com/docs/resources/self_hosting/sharding/overview
26. Elasticsearch vs Qdrant (vs Meilisearch): Which search solution actually fits your needs in 2025?, accessed March 30, 2026, <https://www.meilisearch.com/blog/elasticsearch-vs-qdrant>
27. Elasticsearch vs Pinecone vs Meilisearch: Which Fits? [2025], accessed March 30, 2026, <https://www.meilisearch.com/blog/elasticsearch-vs-pinecone>
28. Meilisearch vs Elasticsearch, accessed March 30, 2026, <https://www.meilisearch.com/blog/meilisearch-vs-elasticsearch>
29. Welcome to Meilisearch - Meilisearch Documentation, accessed March 30, 2026, https://www.meilisearch.com/docs/getting_started/overview
30. Benchmarking Performance: Elasticsearch vs Competitors | by Gigasearch Engineering, accessed March 30, 2026, <https://medium.com/gigasearch/benchmarking-performance-elasticsearch-vs-competitors-d4778ef75639>
31. Indexing optimization guide - Meilisearch, accessed March 30, 2026, <https://www.meilisearch.com/blog/indexing-optimization-guide>
32. Running Meilisearch in production, accessed March 30, 2026, https://www.meilisearch.com/docs/resources/self_hosting/deployment/running_production
33. Using Meilisearch with Docker, accessed March 30, 2026, https://www.meilisearch.com/docs/resources/self_hosting/getting_started/docker
34. Configure Meilisearch at launch, accessed March 30, 2026, https://www.meilisearch.com/docs/resources/self_hosting/configuration/overview
35. Olam transforms agriculture business with Nutanix Cloud Infrastructure, accessed March 30, 2026, <https://www.nutanix.com/company/customers/olam>
36. AWS Marketplace: Meilisearch - Amazon.com, accessed March 30, 2026, <https://aws.amazon.com/marketplace/pp/prodview-7ougkqatzyr7i>
37. AWS Marketplace - Meilisearch, accessed March 30, 2026, <https://www.meilisearch.com/blog/aws-marketplace>
38. Progress You Can Measure: Nutanix Unified Storage Pushes AI Performance Further, accessed March 30, 2026, <https://www.nutanix.com/blog/nutanix-unified-storage-pushes-ai-performance-further>
39. Nutanix AHV vs. VMware ESXi: Making an Informed Decision - Veeam, accessed March 30, 2026, <https://www.veeam.com/blog/nutanix-ahv-vs-vmware-esxi.html>
40. Enterprise and Community editions - Meilisearch Documentation, accessed March

- 30, 2026,
https://www.meilisearch.com/docs/resources/self_hosting/enterprise_edition
41. Meilisearch roadmap roundup: where we're heading, accessed March 30, 2026,
<https://www.meilisearch.com/blog/2026-march-roadmap>
 42. Horizontal scaling with sharding - Meilisearch, accessed March 30, 2026,
<https://www.meilisearch.com/blog/horizontal-scaling-with-sharding>
 43. Best Practices for Extracting SAP Data into Databricks using the ODP Framework, accessed March 30, 2026,
<https://community.databricks.com/t5/technical-blog/navigating-the-sap-data-ocean-demystifying-sap-data-extraction/ba-p/94617>
 44. ODP-based Data Extraction via OData - SAP Help Portal, accessed March 30, 2026,
https://help.sap.com/docs/SAP_BW4HANA/107a6e8a38b74ede94c833ca3b7b6f51/584ff8f21d7e40b1be35b5294cfff194.html
 45. How to Enable OData Services in SAP S/4HANA, accessed March 30, 2026,
<https://help.sap.com/docs/advanced-financial-closing/administration/how-to-enable-odata-services-in-sap-s4hana>
 46. A basic understanding of the analytics architecture in SAP S/4HANA, accessed March 30, 2026,
<https://learning.sap.com/courses/configuring-asset-accounting-in-sap-s4hana/a-basic-understanding-of-the-analytics-architecture-in-sap-s-4hana>
 47. APIs on SAP Business Accelerator Hub - SAP Help Portal, accessed March 30, 2026,
https://help.sap.com/docs/SAP_S4HANA_ON-PREMISE/8308e6d301d54584a33cd04a9861bc52/1e60f14bdc224c2c975c8fa8bcfd7f3f.html
 48. Extracting SAP data using OData - Part 4 - Handling large volumes of data, accessed March 30, 2026,
<https://techcommunity.microsoft.com/blog/azuresynapseanalyticsblog/extracting-sap-data-using-odata---part-4---handling-large-volumes-of-data/2849727>
 49. Mindsprint BTP Services_Fastest way to futureproof your SAP investments and operations, accessed March 30, 2026,
https://assets.ctfassets.net/uwkplyxd6z8/72w1FXkcmj07bGDmLKHP3v/750a7a303873d84c7d04211360e3d229/Mindsprint_BTP_Services_Fastest_way_to_futureproof_your_SAP_investments_and_operations_-_A_Mindsprint_Brochure.pdf
 50. SAP Integration Suite | Integration Software, accessed March 30, 2026,
<https://www.sap.com/products/technology-platform/integration-suite.html>
 51. Integration Suite - SAP Discovery Center Service, accessed March 30, 2026,
<https://discovery-center.cloud.sap/serviceCatalog/integration-suite>
 52. Good practices - Meilisearch Documentation, accessed March 30, 2026,
https://www.meilisearch.com/docs/getting_started/good_practices
 53. How do I access OData service using the SAP S/4HANA Cloud SDK in existing application?, accessed March 30, 2026,
<https://stackoverflow.com/questions/52617170/how-do-i-access-odata-service-using-the-sap-s-4hana-cloud-sdk-in-existing-applic>
 54. Using Python to get data from s4 hana cloud by oda... - SAP Community,

- accessed March 30, 2026,
<https://community.sap.com/t5/enterprise-resource-planning-blog-posts-by-sap/using-python-to-get-data-from-s4-hana-cloud-by-odata-and-use-python-pandas/ba-p/13512789>
55. meilisearch/integration-guides - GitHub, accessed March 30, 2026,
<https://github.com/meilisearch/integration-guides>
 56. Official SDKs and libraries - Meilisearch Documentation, accessed March 30, 2026, <https://www.meilisearch.com/docs/resources/help/sdks>
 57. Automate Elasticsearch Data Replication to SAP HANA - CData Software, accessed March 30, 2026,
<https://www.cdata.com/kb/tech/elasticsearch-sync-saphana.rst>
 58. Elasticsearch Connector (CData Connect Cloud) - SAP, accessed March 30, 2026,
<https://www.sap.com/products/technology-platform/partners/cdata-software-inc-elasticsearch-connector-cdata-connect-cloud.html>
 59. SAP S/4HANA OData Connector Release Notes - MuleSoft Documentation, accessed March 30, 2026,
<https://docs.mulesoft.com/release-notes/connector/sap-s4-hana-connector-release-notes-mule-4>
 60. Flutter vs React Native vs Ionic [Developer's perspective] - Ulam Labs, accessed March 30, 2026, <https://www.ulam.io/blog/flutter-vs-react-native-vs-ionic>
 61. From Native to React Native to Flutter - Zerodha Tech Blog, accessed March 30, 2026, <https://zerodha.tech/blog/from-native-to-react-native-to-flutter/>
 62. Multi-tenancy guide - Meilisearch, accessed March 30, 2026,
<https://www.meilisearch.com/blog/multi-tenancy-guide>
 63. Enterprise search: a comprehensive guide - Meilisearch, accessed March 30, 2026, <https://www.meilisearch.com/blog/enterprise-search>
 64. How to implement RBAC with Meilisearch, accessed March 30, 2026,
<https://www.meilisearch.com/blog/role-based-access-guide>
 65. How to use Meilisearch in your multi-tenant application, accessed March 30, 2026,
<https://www.meilisearch.com/blog/multi-tenancy>
 66. Meilisearch is too slow · Issue #11 · Kerollmops/blog - GitHub, accessed March 30, 2026, <https://github.com/Kerollmops/blog/issues/11>
 67. About two major LMDB optimization tricks #4793 - GitHub, accessed March 30, 2026, <https://github.com/meilisearch/meilisearch/issues/4793>
 68. Nutanix vs VMware Showdown Performance Cost and Features - Coherent Market Insights, accessed March 30, 2026,
<https://www.coherentmarketinsights.com/blog/information-and-communication-technology/nutanix-vs-vmware-showdown-performance-cost-and-features-2624>